

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

Procesamiento y Optimización de Consultas
Práctico N.º 5

Alumno: Tomás Rodeghiero
Año: 2026

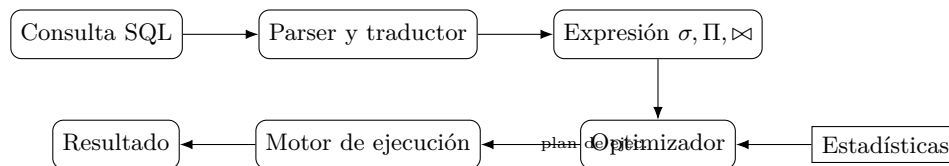
Índice

1. Visión general	2
2. Costos: cómo se mide la eficiencia	2
3. Algoritmos para la operación de selección	3
3.1. Selecciones simples (un solo predicado)	3
3.2. Selecciones complejas (conjunción y disyunción)	4
4. Ordenamiento	5
5. Algoritmos para la operación de join	5
5.1. Nested-loop join	5
5.2. Indexed nested-loop join	6
5.3. Merge-join	6
5.4. Hash-join	6
6. Evaluación de expresiones: materialización vs pipelining	6
7. Optimización: generación de expresiones equivalentes	7
7.1. Reglas más usadas (Teórico 7)	7
7.2. Intuición y ejemplo	7
7.3. Orden de joins	8
8. Optimización basada en costos vs heurística	8
9. Estadísticas para estimación de costos	8
9.1. Variables que mantiene el catálogo	8
9.2. Estimación de la selección	9
9.3. Estimación del tamaño de un join	9
9.4. Estimación de otras operaciones	9
10. Técnicas adicionales de optimización	10
10.1. Subqueries anidados	10
10.2. Vistas materializadas	10
11. Cómo se conecta con PostgreSQL (Práctico 5)	10
12. Conclusión	11

1. Visión general

Cuando un motor de base de datos recibe una consulta SQL no la ejecuta literalmente: la procesa en **tres etapas**, cada una con responsabilidades bien delimitadas:

1. **Parsing y traducción.** Se verifica la sintaxis y que las relaciones y columnas referenciadas existan. Luego el SQL se traduce a una expresión del *álgebra relacional*. Si la consulta usa *vistas*, la traducción reemplaza la vista por su definición.
2. **Optimización.** A partir de la expresión inicial se generan *expresiones equivalentes* (mismo resultado, distinto orden o forma) y, por cada una, varios *planes de ejecución* (qué algoritmo concreto se usa para cada operador). El optimizador estima costos y elige el plan más barato.
3. **Evaluación.** El motor toma el plan elegido y lo ejecuta contra los datos en disco, devolviendo el resultado.



El *punto importante* es que la diferencia de costo entre dos planes equivalentes puede ser enorme (varios órdenes de magnitud). Por eso la optimización no es un lujo: es lo que hace que SQL sea usable a escala.

2. Costos: cómo se mide la eficiencia

El costo de una consulta es básicamente el tiempo de respuesta. En la práctica intervienen muchos factores (CPU, RAM, red, disco), pero el Teórico 6 simplifica: **el costo dominante es el acceso a disco**, y se calcula contando dos cosas:

- número de *bloques transferidos* (lectura/escritura), con tiempo unitario t_T ;
- número de *búsquedas* (*seeks*, posicionamientos del cabezal o cambio de partición), con tiempo unitario t_b .

Si una operación transfiere b bloques y hace s búsquedas, su costo estimado es

$$\text{Costo} = b \cdot t_T + s \cdot t_b.$$

Valores de referencia (Teórico 6). $t_b = 4$ ms, $t_T = 0,1$ ms, bloque de 4 KB, velocidad de transferencia 40 MB/s. Son valores “académicos”; en SSD modernos t_b es mucho menor ($t_b \approx 0$ en NVMe), pero el modelo sigue siendo correcto: *las búsquedas son caras frente a las transferencias contiguas*.

Variables del catálogo que aparecen en los costos.

- n_r : cantidad de tuplas de la relación r .
- b_r : cantidad de bloques que contienen tuplas de r .
- t_r : tamaño promedio de una tupla.
- f_r : *factor de bloque*, tuplas por bloque ($b_r = \lceil n_r / f_r \rceil$).
- $V(A, r)$: cantidad de valores distintos del atributo A en r .
- h_i : altura del índice (B+tree).

Estas variables son las mismas que cualquier motor real mantiene en su catálogo (en PostgreSQL: `pg_class.reltuples`, `relpages`, `pg_stats.n_distinct`, etc.).

3. Algoritmos para la operación de selección

La selección $\sigma_\theta(r)$ es el operador más estudiado porque interviene en casi todas las consultas. El Teórico 6 lista once algoritmos identificados como A1... A11; el optimizador elige uno según haya o no haya índices y según la forma de θ (igualdad, rango, conjunción, disyunción).

3.1. Selecciones simples (un solo predicado)

A1: búsqueda lineal

Lee bloque por bloque, descarta los que no cumplen el predicado. Es el “algoritmo universal”: siempre se puede aplicar.

$$\text{Costo}_{A1} = b_r \cdot t_T + t_b.$$

Si la condición es de *igualdad por clave* (a lo sumo una tupla cumple), la búsqueda se corta en cuanto aparece: en promedio $(b_r/2) t_T + t_b$.

Por qué importa. A1 es el plan de referencia. Cualquier otro plan vale la pena sólo si es más barato que A1. Por eso, cuando la selectividad es muy alta, los motores eligen Seq Scan aunque haya índice disponible: leer todo en orden es más barato que saltar de bloque en bloque.

A2: búsqueda binaria

Aplicable si el archivo está *ordenado* por el atributo y la condición es de igualdad: localiza la primera tupla con costo $\lceil \log_2 b_r \rceil \cdot (t_T + t_b)$ y luego transfiere los bloques contiguos.

A3, A4, A5: igualdad con índice

- **A3** — igualdad sobre *índice de clave primaria o único*. A lo sumo una tupla. Costo $(h_i + 1)(t_T + t_b)$.
- **A4** — igualdad sobre *índice primario no único* (atributo no clave, pero el archivo está ordenado por él). Las tuplas que cumplen están en bloques contiguos: $h_i(t_T + t_b) + t_b + b \cdot t_T$, con b = bloques que contienen tuplas que cumplen.
- **A5** — igualdad sobre *índice secundario*. Si el atributo no es único pueden recuperarse varias tuplas, una I/O por tupla en el peor caso: $(h_i + n)(t_T + t_b)$.

Intuición. A3 es el caso ideal (devuelve una sola tupla y baja el árbol del índice una sola vez). A5 paga por cada tupla devuelta porque puede estar en un bloque distinto: si la selectividad es muy baja, A5 puede volverse peor que A1.

A6 y A7: rango con índice

- **A6** — rango sobre *índice primario*. Para $A \geq v$ se busca la primera tupla con el índice y se sigue secuencialmente. Para $A \leq v$ se arranca desde el inicio del archivo y se para en la primera tupla que viole la condición; el índice no agrega valor en este último caso.
- **A7** — rango sobre *índice secundario*. Se recorren las hojas del índice para conseguir los punteros y, por cada puntero, se hace una I/O.

Cuándo conviene un índice secundario sobre uno primario. El índice primario garantiza que las tuplas devueltas vienen en bloques contiguos. El secundario, no: cada tupla puede estar en un bloque distinto y obliga a *tantas* I/O como tuplas, lo que rápidamente se vuelve más caro que leer el archivo entero (A1). Por eso, para rangos amplios, los motores suelen preferir Seq Scan al *Index Scan* con índice secundario.

3.2. Selecciones complejas (conjunción y disyunción)

A8: conjunción usando un solo índice

Para $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$:

1. Elegir la condición θ_i que dé el plan más barato según A1–A7.
2. Recuperar las tuplas que cumplen θ_i .
3. Testear el resto de las condiciones en memoria.

Es la estrategia “elegir la condición más restrictiva”.

A9: índice multiclave

Si existe un índice compuesto que cubre exactamente la combinación de predicados, se usa directamente. Ejemplo: índice (a,b) para $\sigma_{a=v_1 \wedge b=v_2}(r)$.

A10: intersección de identificadores

Para conjunciones donde *cada* condición tiene un índice utilizable:

1. Obtener el conjunto de punteros P_i que cumple θ_i usando su índice (lógica A2–A7).
2. Calcular $P_1 \cap P_2 \cap \dots \cap P_n$.
3. Recuperar sólo las tuplas referenciadas por la intersección.

Es el algoritmo subyacente a los *Bitmap Index Scan* de PostgreSQL.

A11: unión de identificadores (disyunción)

Para $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$ con un índice para cada θ_i :

1. Obtener P_i con el índice de θ_i .
2. Calcular $P_1 \cup P_2 \cup \dots \cup P_n$ (sin duplicados).
3. Recuperar las tuplas referenciadas.

Si alguna condición no tiene índice, no aplica y hay que caer a A1.

Resumen de la familia A1–A11.

Algoritmo	Aplicabilidad	Idea
A1	Siempre	Búsqueda lineal
A2	Archivo ordenado, igualdad	Búsqueda binaria
A3	Igualdad, índice único	Una sola tupla
A4	Igualdad, índice primario no único	Tuplas contiguas
A5	Igualdad, índice secundario	I/O por tupla
A6	Rango, índice primario	Recorrido secuencial
A7	Rango, índice secundario	Recorrer hojas + I/O
A8	Conjunción + un índice	Filtrar el resto en RAM
A9	Conjunción + índice multiclave	Un solo acceso
A10	Conjunción + varios índices	Intersección de RIDs
A11	Disyunción + índices	Unión de RIDs

4. Ordenamiento

Por qué importa. Muchas operaciones (group by, distinct, merge join, order by) requieren entrada ordenada. Cómo se ordena depende de si la relación entra en memoria:

- Si la relación cabe en RAM: cualquier algoritmo clásico (*quicksort*, *heapsort*, ...).
- Si no cabe: **External Sort-Merge**.
- Si ya existe un índice sobre el atributo a ordenar: alcanza con recorrerlo, una I/O por tupla. Conviene sólo si la cantidad de tuplas es chica (de lo contrario es más barato ordenar de cero).

External Sort-Merge. Con M bloques de memoria disponibles:

1. **Fase de creación de corridas:** leer M bloques a la vez, ordenarlos en memoria y escribir cada bloque ordenado como una *corrida* S_i a disco. Quedan $N = \lceil b_r/M \rceil$ corridas ordenadas pero no entre sí.
2. **Fase de merge N -way:** si $N < M$, se mezclan las N corridas con N buffers de entrada y 1 de salida. Si $N \geq M$, se hacen varias pasadas, mezclando $M - 1$ corridas en cada una y reduciendo el número total por ese factor.

5. Algoritmos para la operación de join

El join $r \bowtie_{\theta} s$ es la operación con más algoritmos a elegir porque es la más cara y la que más se beneficia de la elección correcta.

5.1. Nested-loop join

```

for each tupla  $t_r$  en  $r$  do
  for each tupla  $t_s$  en  $s$  do
    si  $\theta(t_r, t_s)$  es verdadero, emitir  $t_r \cdot t_s$ 

```

Es el más simple y se aplica a cualquier condición θ (no sólo igualdad). Costo del peor caso:

$$n_r \cdot b_s + b_r \text{ bloques transferidos} + n_r + b_r \text{ búsquedas.}$$

Ejemplo del teórico. Con $n_{cli} = 10\,000$, $b_{cli} = 400$, $n_{fac} = 5\,000$, $b_{fac} = 100$:

- **factura** como externa: $5000 \cdot 400 + 100 = 2,000,100$ bloques transferidos.

- `cliente` como externa: $10000 \cdot 100 + 400 = 1,000,400$.

Conclusión: poner como *externa* la relación que minimice el producto $n_{ext} \cdot b_{int}$; idealmente, la más chica como interna entera en memoria.

Block nested-loop. Variante que carga *bloques* de r en lugar de tuplas individuales y compara contra todas las de s . Reduce muchísimo el número de búsquedas. Si la relación más chica entra en memoria el costo cae a $b_r + b_s$ con 2 búsquedas.

5.2. Indexed nested-loop join

Si existe un índice sobre el atributo del join en la relación interna s , por cada tupla de r se busca directamente en el índice. Útil para joins selectivos o cuando s es grande pero el predicado de igualdad recupera pocas tuplas.

5.3. Merge-join

Aplicable a *equi-joins* y joins naturales. Funciona así:

1. Ordenar ambas relaciones por los atributos del join (si todavía no lo están).
2. Recorrer las dos en paralelo como un *merge*: avanzar la relación cuyo valor sea menor; cuando coinciden, emitir el cruce de tuplas con ese valor.

Cada relación se recorre *una vez*. Si las relaciones ya están ordenadas (por ejemplo, por índice clusterizado o por una operación previa que dejó el orden), Merge-Join es muy barato.

5.4. Hash-join

Aplicable a joins de igualdad y naturales:

1. Elegir una función hash h sobre los atributos del join.
2. Particionar r en buckets $r_0, r_1, \dots, r_n$ según h y análogamente s en $s_0, \dots, s_n$. La clave: tuplas de r_i sólo pueden cruzarse con tuplas de s_i (porque tienen el mismo hash).
3. Por cada par (r_i, s_i) , hacer el join en memoria (típicamente *nested loop* sobre la partición chica con tabla hash sobre la otra).

Es el algoritmo que más usan los motores modernos cuando ninguna relación viene ordenada. Es el caso por defecto cuando se hace NATURAL JOIN sin índices ni ORDER BY (como pasa en el Ejercicio 5.e).

Comparación práctica.

Algoritmo	Condiciones soportadas	Cuándo conviene
Nested-loop	Cualquiera	Una relación muy chica o como fallback
Nested-loop (bloques)	Cualquiera	Relación chica cabe en RAM
Indexed nested-loop	Cualquiera + índice	Join muy selectivo en la interna
Merge-join	Igualdad / natural	Entradas ya ordenadas
Hash-join	Igualdad / natural	Cualquier tamaño sin orden previo

6. Evaluación de expresiones: materialización vs pipelining

Hasta aquí se vieron operadores individuales. Una consulta real es un árbol de operadores. Hay dos estrategias para combinarlos:

Materialización. Cada subexpresión se evalúa por completo y su resultado se escribe en disco; el operador siguiente lee desde ese resultado intermedio. Es simple pero paga el costo de escribir y leer la relación intermedia entera.

Pipelining. Las tuplas que va produciendo un operador se pasan inmediatamente al operador siguiente, sin escribir el intermedio en disco. Es como un *stream* de tuplas. Es lo que hacen los motores modernos cuando el operador siguiente es *tuple-at-a-time* (selección, proyección, nested-loop, hash-join probe).

Quién bloquea el pipeline. Hay operadores que necesitan ver *toda* su entrada antes de producir el primer resultado: por ejemplo, sort, hash-join en su fase de *build*, agrupación. Estos son los que fuerzan materialización en el árbol.

7. Optimización: generación de expresiones equivalentes

El optimizador toma la expresión inicial (resultado del traductor) y la reescribe usando **reglas de equivalencia**: dos expresiones son equivalentes si producen el mismo resultado (en SQL, el mismo *multiset*) para cualquier instancia de la base de datos.

7.1. Reglas más usadas (Teórico 7)

1. **R1 — Cascada de selecciones.** $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$.
2. **R2 — Conmutatividad de selecciones.** $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$.
3. **R3 — Cascada de proyecciones.** Sólo importa la proyección externa: $\Pi_{L_1}(\Pi_{L_2}(\dots(E))) = \Pi_{L_1}(E)$.
4. **R4 — Selección sobre producto/theta-join.** $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$.
5. **R5 — Conmutatividad de join.** $E_1 \bowtie E_2 = E_2 \bowtie E_1$.
6. **R6 — Asociatividad de join.** $(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$.
7. **R7 — Push-down de selección sobre join.** Si θ_0 involucra sólo atributos de E_1 : $\sigma_{\theta_0}(E_1 \bowtie E_2) = \sigma_{\theta_0}(E_1) \bowtie E_2$. Si θ_1 usa sólo E_1 y θ_2 sólo E_2 : $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie E_2) = \sigma_{\theta_1}(E_1) \bowtie \sigma_{\theta_2}(E_2)$.
8. **R8 — Push-down de proyección sobre join.** $\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2)$ se distribuye, conservando en cada operando los atributos que pertenezcan a $L_1 \cup L_2$ o aparezcan en θ .
9. **R9–R12 — Operaciones de conjuntos.** Unión e intersección son conmutativas y asociativas; la resta no. La selección distribuye sobre unión, intersección y resta.

7.2. Intuición y ejemplo

Por qué empujar selecciones. Si una selección elimina el 99% de las tuplas, conviene aplicarla antes de cualquier join porque achica el tamaño del operando que entra al join. El join trabaja sobre menos tuplas y produce menos tuplas.

Ejemplo (Teórico 7).

$$\Pi_{desc}(\sigma_{nombre="Super\ Vea" \wedge precio_venta < 10}(proveedor \bowtie provee \bowtie articulo))$$

Aplicando R1 (cascada), R7 (push-down) y R5/R6 (reordenar) se llega a:

$$\Pi_{desc}(\sigma_{nombre="Super\ Vea"}(proveedor) \bowtie \sigma_{precio_venta < 10}(provee) \bowtie articulo)$$

Ahora cada selección actúa sobre la tabla base más chica y los joins trabajan sobre operandos ya reducidos.

7.3. Orden de joins

Para tres relaciones, asociatividad da dos planes: $(r_1 \bowtie r_2) \bowtie r_3$ y $r_1 \bowtie (r_2 \bowtie r_3)$. Si $r_2 \bowtie r_3$ resulta mucho mayor que $r_1 \bowtie r_2$, el primer plan es preferible (resultado intermedio más chico). Para n relaciones hay $\frac{(2(n-1))!}{(n-1)!}$ órdenes posibles; con $n = 10$, casi 18 mil millones. Por eso el optimizador usa **programación dinámica**: calcula el mejor plan para cada subconjunto de relaciones una sola vez y los combina.

8. Optimización basada en costos vs heurística

Los optimizadores reales combinan dos enfoques:

Optimización basada en costos. Enumerar (parte de) las expresiones equivalentes posibles, estimar el costo de cada plan usando las estadísticas del catálogo y elegir el más barato. Es preciso pero caro (espacio de planes enorme).

Optimización heurística. Aplicar reglas que *típicamente* mejoran el plan sin estimar costos para cada caso. Las heurísticas clásicas son:

- realizar las selecciones tan pronto como sea posible (reducir cantidad de tuplas),
- realizar las proyecciones tan pronto como sea posible (reducir ancho de tupla),
- realizar primero los joins/selecciones más restrictivos (los que den resultados intermedios más chicos).

Los motores reales hacen primero una pasada heurística (push-down, reescritura de vistas, eliminación de subqueries) y luego una pasada basada en costos sobre el espacio reducido.

Atención. Las heurísticas mejoran “en general”, no siempre. Empujar una selección puede ser contraproducente si la condición es muy costosa de evaluar (por ejemplo, una función definida por usuario), o si le quita ordenamiento útil al operador siguiente.

9. Estadísticas para estimación de costos

Para que la optimización por costos funcione, el optimizador necesita *estimar* cuánto cuesta cada plan sin ejecutarlo. Esto se basa enteramente en las estadísticas del catálogo.

9.1. Variables que mantiene el catálogo

- n_r, b_r, t_r, f_r por relación.
- $V(A, r)$: cantidad de valores distintos del atributo A en r .
- $\min(A, r)$ y $\max(A, r)$.
- **Histogramas** sobre los atributos más consultados, típicamente equi-depth: dividen el rango de valores en buckets con igual cantidad de tuplas. Sirven para estimar selectividad de rangos cuando los valores no están distribuidos uniformemente.
- **Most Common Values (MCV)**: lista de valores más frecuentes con su frecuencia. Sirven para estimar igualdades cuando el valor pedido aparece en la lista.

9.2. Estimación de la selección

Igualdad: $\sigma_{A=v}(r)$. Si A no es clave: $n_r/V(A, r)$ tuplas (asume distribución uniforme). Si A es clave: a lo sumo 1 tupla. Si hay MCV y $v \in \text{MCV}$: usar la frecuencia exacta.

Rango: $\sigma_{A \leq v}(r)$. Si no hay histograma: aproximar linealmente entre mín y máx:

$$c = n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}.$$

Con histograma equi-depth: sumar bucket por bucket, interpolando el bucket final.

Conjunción: $\sigma_{\theta_1 \wedge \dots \wedge \theta_n}(r)$. Asumiendo *independencia* entre las condiciones:

$$n_r \cdot \frac{s_1 \cdot s_2 \cdots s_n}{n_r^n},$$

donde $s_i = |\sigma_{\theta_i}(r)|$. La probabilidad s_i/n_r se llama *selectividad* de θ_i .

Disyunción: $\sigma_{\theta_1 \vee \dots \vee \theta_n}(r)$. Por inclusión-exclusión, asumiendo independencia:

$$n_r \cdot (1 - (1 - s_1/n_r)(1 - s_2/n_r) \cdots (1 - s_n/n_r)).$$

Negación: $\sigma_{\neg \theta}(r)$. $n_r - |\sigma_{\theta}(r)|$.

9.3. Estimación del tamaño de un join

Producto cartesiano. $|r \times s| = n_r \cdot n_s$.

Si $R \cap S = \emptyset$. $r \bowtie s$ degenera en $r \times s$: $|r \bowtie s| = n_r \cdot n_s$.

Si el atributo común es clave en una de las relaciones. Cada tupla de la otra matchea con a lo sumo una: $|r \bowtie s| \leq \min(n_r, n_s)$.

Si el atributo común no es clave en ninguna de las dos. Dos estimaciones posibles, dependiendo del lado de la división:

$$\frac{n_r \cdot n_s}{V(A, r)} \quad \text{o} \quad \frac{n_r \cdot n_s}{V(A, s)}.$$

Se toma la *menor* como mejor estimación. Si hay histogramas se aplica la misma fórmula por celda.

9.4. Estimación de otras operaciones

- Proyección $\Pi_A(r)$: tamaño $\leq V(A, r)$ (con eliminación de duplicados).
- Unión $r \cup s$: cota superior $|r| + |s|$.
- Intersección $r \cap s$: cota superior $\min(|r|, |s|)$.
- Diferencia $r - s$: cota superior $|r|$.

10. Técnicas adicionales de optimización

10.1. Subqueries anidados

SQL trata conceptualmente a un subquery del **where** como una función que se evalúa una vez por cada tupla del nivel externo (*evaluación correlacionada*). En el peor caso esto multiplica el costo por n_r .

Los optimizadores intentan reescribir los subqueries como joins:

```
-- Forma anidada
SELECT descripcion FROM articulo
WHERE EXISTS (SELECT 1 FROM provee
              WHERE provee.nArt = articulo.nArt);

-- Forma equivalente con semijoin / join (mejor plan)
SELECT DISTINCT a.descripcion
FROM articulo a JOIN provee p ON p.nArt = a.nArt;
```

La forma con join expone más estructura al optimizador (puede usar hash o merge join, índices, etc.).

10.2. Vistas materializadas

Una **vista materializada** es una vista cuyo contenido se calcula y se almacena físicamente. Mejora la performance de consultas costosas a costa de mantener actualizado el resultado.

```
CREATE VIEW precioMenor (nArt, precio_venta) AS
SELECT nArt, min(precio_venta)
FROM provee
GROUP BY nArt;
```

Mantenimiento. Tres estrategias:

- Recomputar cada cierto tiempo (*batch*, p.ej. cada noche).
- Recomputar al **commit** (*refresh fast/on commit* en Oracle).
- **Mantenimiento incremental:** aplicar los cambios sobre la vista a medida que cambian las tablas base, usando *triggers* o logs de cambio. Es la opción más eficiente pero la más compleja.

11. Cómo se conecta con PostgreSQL (Práctico 5)

Todo lo anterior se vuelve operativo en el Ejercicio 5 del práctico:

- **Catálogo:** `pg_class`, `pg_attribute`, `pg_stats` contienen n_r , b_r , $V(A, r)$, histogramas y MCVs de las tablas `persona`, `recurso` y `seguimiento_acceso`.
- **Algoritmos de selección:** el plan inicial sobre `seguimiento_acceso` filtrando por `fecha_hora_entrada` es un *Seq Scan* (A1). Al crear un B+tree sobre la columna y correr `ANALYZE`, el plan cambia a *Bitmap Index Scan + Heap Scan* (variantes de A5/A7 según la selectividad).
- **Algoritmos de join:** NATURAL JOIN sin orden previo arranca con *Hash Join*. Creando un índice sobre la columna del join y agregando `ORDER BY` por la clave de join, el planner cambia a *Merge Join*. Si la decisión no cambia se puede usar `SET enable_hashjoin = off` para verificación académica.

- **Estimación:** el comando `EXPLAIN (ANALYZE, BUFFERS)` muestra junto al plan el costo *estimado* y el costo *real*, permitiendo comparar las estimaciones del optimizador con la ejecución efectiva.

12. Conclusión

Procesar una consulta SQL es, en el fondo, un problema de búsqueda en un espacio de planes equivalentes. El motor traduce el SQL a álgebra relacional, genera planes con reglas de equivalencia, estima cada uno con las estadísticas del catálogo y ejecuta el más barato.

Las ideas centrales para retener son:

- el costo dominante es el acceso a disco (transferencias y búsquedas);
- los algoritmos de selección (A1–A11) y de join se eligen según los índices disponibles y la selectividad;
- las reglas de equivalencia permiten reescribir la expresión sin cambiar su resultado, y suelen mejorar el costo cuando empujan selecciones y proyecciones a las hojas;
- las estadísticas del catálogo son el insumo crítico: sin ellas, el optimizador estima mal y elige peor;
- ningún algoritmo es mejor siempre: la decisión depende de la forma del predicado, del tamaño relativo de las relaciones, de los índices y del estado de la memoria.